

第19讲 练习题与答案

Inference: 按下“发送”以后，大语言模型内部到底发生什么？

使用建议

先独立完成前两页，再看答案。答案重点解释“为什么”，不要只背术语。

1. 概念判断

“Inference 就是在用户问问题时继续训练模型，只不过只训练几步。”这句话哪里错？

2. Prefill

为什么一个 2000-token prompt 不需要像输出那样严格一个 token 一个 token 串行处理？

3. Decode

为什么生成第 50 个 token 以前，通常必须先知道第 49 个 token 实际生成了什么？

4. Logits

模型给“模型”“系统”“天气”三个候选的 logits 分别是

4.2、3.7、1.2。为什么不能直接说它们的概率就是 4.2、3.7、1.2？

5. KV Cache

用自己的话解释 KV Cache 保存什么，以及它为什么可以减少重复计算。

6. Cache vs Memory

为什么 KV Cache 不能直接等同于“Agent 长期记忆”？

7. Latency

一个请求有 50k input tokens，但只输出 10 个 token。另一个请求只有 200 input tokens，但输出 3000 个 token。两者可能分别在哪个阶段更重？

8. Streaming

为什么 Streaming 可以让用户更早看到输出，但不保证整个回答更快算完？

9. Agent Runtime

“ 37×91 是多少？” Agent 选择 calculator 工具。请写出一次 tool-call loop 的五个主要步骤。

10. 科研设计

比较 Single-Agent 与 Multi-Agent 时，如果只报告 accuracy，不报告 token / calls / latency，会隐藏什么问题？

答案与解释

1.

Inference 通常不做训练所需的反向传播和大规模参数更新，而是使用已有参数做前向计算并生成 token。RAG/Memory 可以改变输入，但不等于训练参数。

2.

Prompt 中所有 token 已经确定。Causal Mask 只限制位置 i 不能读取未来位置，并不要求这些已知位置按时间一个个计算；可以把它们组成矩阵并行完成大量 Q/K/V 与 FFN 运算。

3.

第49个 token 会成为第50步的前缀的一部分。若第49步可能采样出不同 token，第50步的条件分布也会变化，因此真实自回归序列存在串行依赖。

4.

Logit 是未归一化实数分数，可以为任意实数，和为 1 没有关系。需要经过 Softmax 等变换才得到概率分布。

5.

缓存每层 Attention 对历史 token 计算出的 Key / Value。新 token 到来时，只为新 token 计算新 K/V 并追加；不再对所有过去 token 重新做 K/V 投影。

6.

KV Cache 是当前推理会话的计算中间量，主要服务于加速 Attention；长期记忆通常是可跨轮次保存、检索和选择的信息结构，两者生命周期和语义角色不同。

7.

50k 输入、10 输出：Prefill 和初始 KV Cache 压力更突出；200 输入、3000 输出：Decode steps、持续 cache 占用和 end-to-end latency 更突出。真实瓶颈还与硬件和 serving stack 有关。

8.

Streaming 把已经算出的增量 token 立刻传给客户端，缩短“看到第一个结果”的等待；但模型仍需要完成后续所有 decode steps，因此底层总计算量不一定减少。

9.

LLM 生成结构化 tool call $\rightarrow$ runtime 解析 $\rightarrow$ 执行 calculator $\rightarrow$ 工具结果作为 tool message 写回上下文 $\rightarrow$ 再次调用 LLM 继续生成答案。

10.

可能把更多 test-time compute 当成协作收益。Multi-Agent 可能用了更多调用、更长上下文、更多输出 token、Judge 调用和更高成本，因此要做 compute-matched baseline。

加餐：把术语连成一句话

试着不看讲义，用下面这些词讲出一次完整 inference：

Chat Template -> Tokenizer -> Token IDs -> Embedding -> Prefill -> KV Cache -> LM Head -> Logits -> Sampling -> Decode -> EOS

参考表述：聊天系统先把 system/user/history 按模板拼接并 tokenize；模型对 prompt 做 prefill、建立 KV cache；最后位置 hidden state 通过 LM head 产生 logits；runtime 按解码策略选出 next token；把 token 追加到序列后继续 decode，直到 EOS 或其他停止条件。